

helixcode_testsuite_triage

HelixCode go test ./... Failure Triage (no-infra run)

Run: go test -count=1 ./... in helix_code/ WITHOUT make test-infra-up (no real PostgreSQL / Redis / Ollama / verifier server / running HelixCode server). **Result:** TEST_EXIT=1 — 178 ok, 24 FAIL (primary log qa-results/helixcode_unittest_20260623_220246.log). **Triage date:** 2026-06-23 · read-only, no code touched, suite NOT re-run. **Honest boundary (§11.4.6):** categories below are derived from the log + test source. Items marked **UNCONFIRMED** need a make test-infra-up re-run to confirm.

A sibling run (..._220256.log) under heavier parallel load failed a DIFFERENT, LARGER set (added internal/deployment GPU-probe tests, internal/tools, tests/e2e/core, extra MCP WS-transport tests). The two runs diverging on which packages fail is itself strong evidence most of these are **ENV/FLAKY-TIMING**, not deterministic product defects.

Failure table (primary log, 24 package-level failures across the named tests)

Package	Failing test(s)	Category	Evidence
internal/agent/subagent	TestSubprocessSpawner_RealHelper_RoundTrip	ENV/FLAKY-TIMING	10.00s t helper
internal/cognee	TestClientHTTP_Chaos_ServerCloseDuringConcurrentLoad	NEEDS-REAL-INFRA	“cognee exceeded
internal/cognee	TestClientAddMemory/AddMemory_Success (+ panic)	NEEDS-REAL-INFRA (+ latent REAL-BUG)	Post h api/v operat deref pa
internal/discovery	TestIsPortReachable	ENV/FLAKY-TIMING	client_t host TC
internal/discovery	TestHealthMonitor_CheckServiceHealth_TCP	ENV/FLAKY-TIMING	health_r “httptest — TCP
internal/mcp	TestStdioTransport_StderrCapture	ENV/FLAKY-TIMING	19.29s; timing
internal/providers/httpclient	TestSharedClient_BurstReuse	ENV/FLAKY-TIMING	0.01s bu
internal/verifier	TestAdapter_GetVerifiedModels_FallbackOnError	REAL-BUG (stale test)	

Package	Failing test(s)	Category	Evidence
			adapter_ requir fallback
internal/ voice	TestVoiceRecorder_StartLaunchesProcess_Guard	ENV/FLAKY- TIMING	recorder missing audio ca
tests/e2e/ phase2	TestBasicIntegration, TestServerCapabilities, TestRealServerIntegration	NEEDS-REAL- INFRA	integrati failed: m running
tests/e2e/ phase3	TestMemorySystemIntegration, TestConversationMemory, TestMemorySearchAndRetrieval, TestMemoryPersistence, TestMemoryAnalytics, TestMemoryPrivacyAndSecurity, TestConcurrentProjectOperations, TestMemoryOptimization, TestResourceCleanup, TestThroughputScalability, TestPhase3Basic, TestPhase3Connectivity	NEEDS-REAL- INFRA	framework map [s t memory
tests/ performance	TestCompetitorBaselineScript_RunsEndToEnd, TestCompetitorBaselineScript_SelfVerifiesMeasuredAgents	REAL-BUG (missing artifact)	script compet does NO
tests/ performance/ scenarios	TestRunner_StableAcrossThreeRuns	ENV/FLAKY- TIMING	“S3 harm (>=35% (HELIX
tests/ regression	TestCriticalPath_ConfigurationLoading/ConfigValidation	UNCONFIRMED (likely REAL- BUG)	critical_ should p memory valida no netw
tests/ regression	TestServerStability	ENV/FLAKY- TIMING	server_t http: / exceed immedia on load

Counts per category (primary log)

- **NEEDS-REAL-INFRA:** 3 packages — internal/cognee (2 tests), tests/e2e/phase2 (3), tests/e2e/phase3 (12). ~17 individual tests.
- **ENV/FLAKY-TIMING:** 8 packages — subagent, discovery (2), mcp, httpclient, voice, performance/scenarios, regression/TestServerStability.
- **REAL-BUG (actionable):** 2 — internal/verifier (stale Len==7), tests/performance (missing competitor_speed_baseline.sh). Plus 1 latent test-robustness bug (cognee panic).
- **UNCONFIRMED:** 1 — tests/regression TestCriticalPath_ConfigurationLoading/ConfigValidation.

REAL-BUG list (the actionable subset — NOT product-breaking for end users)

1. **internal/verifier · TestAdapter_GetVerifiedModels_FallbackOnError** — STALE TEST. `fallback_models.go` defines 8 models (llama-3.2-3b, gpt-4o, claude-3-5-sonnet, mistral-large, gemini-2.5-pro, deepseek-chat, grok-3-fast-beta, **mimo-v2.5-pro**). Test still asserts `require.Len(models, 7)`. The 8th (Xiaomi MiMo) was added without updating the test. **Product is correct; test assertion is stale** — §11.4.120 fix-breaks-its-own-gate / reconcile-the-gate, NOT a CONST-035 product bluff. Fix: update assertion to 8 (and `models[0].Source == "fallback"` still holds).
 2. **tests/performance · TestCompetitorBaselineScript_*** — MISSING ARTIFACT. Both tests reference `helix_code/scripts/testing/competitor_speed_baseline.sh` which does not exist (exit 127). Either the script was never committed or was removed. **Not a product defect**; a test depends on a non-existent helper. §11.4.124-adjacent (missing wiring). Fix: restore/author the baseline script, or mark the test SKIP-OK if the baseline harness is deferred.
 3. **internal/cognee · TestClientAddMemory panic (latent test-robustness bug)**. The *failure* is NEEDS-REAL-INFRA (no cognee backend → POST times out). But after the assertion fails, the test continues and nil-derefs at `cognee_test.go:1143` (SIGSEGV) — the test does not guard against the nil response it just asserted non-nil. Even with infra up this is a fragile test; with infra DOWN it panics the whole package. **Test hardening needed** (guard / require short-circuit), independent of the infra dependency.
-

UNCONFIRMED (needs investigation, possibly REAL-BUG)

- **tests/regression · TestCriticalPath_ConfigurationLoading/ConfigValidation** — `config.NewConfigurationValidator(true).Validate(validCfg)` returns `Valid=false` for a config that looks valid (Version 1.0.0, env=development, server port 8080, DB port 5432, 32-char JWT secret, LLM DefaultProvider="local", MaxTokens=1000). This is a **pure in-memory validator call** — **no network/infra** — so a real-infra re-run will NOT change the verdict. Likely a genuine validator-vs-test divergence (validator now requires a field the fixture omits, or DefaultProvider="local" is no longer accepted). **Disambiguate by:** running just this test and printing `result.Errors`. This session touched config-adjacent areas only via the DeepSeek default-model fix (internal/server passed) — but ConfigValidation could intersect. Recommend the conductor run `go test -run TestCriticalPath_ConfigurationLoading ./tests/regression -v` and read `result.Errors`.
-

CONST-035 / §11.4 anti-bluff assessment

- **NEEDS-REAL-INFRA** tests are NOT bluffs — they correctly FAIL-not-SKIP when the real backend is absent (CONST-050(A): no fakes beyond unit tests; e2e demands real infra). They are running in the wrong environment (no `test-infra-up`), not lying. The `map[status:ok]` e2e message is the test correctly rejecting a non-matching health body / unreachable server — honest failure.

- **ENV/FLAKY-TIMING** tests are test-harness/host issues (port binds, subprocess spawn timing, perf-harness CV noise, audio device), not end-user product defects. They should ideally carry tighter timing tolerances or be load-isolated (§11.4.119 single-resource-owner; §11.4.50 determinism), but none indicates a broken feature.
 - **REAL-BUG (verifier, competitor script)** are test-side defects (stale assertion, missing helper), NOT product bluffs. The verifier *product* returns the full fallback list correctly.
 - **internal/server (DeepSeek default-model fix) PASSED** — the change this session shipped is green.
-

Recommendation

1. **Re-run under** `make test-infra-up` to clear: `internal/cognee`, `tests/e2e/phase2`, `tests/e2e/phase3` (all NEEDS-REAL-INFRA). Expect them to pass once PG/Redis/Ollama/cognee/server are up. This is the bulk (~17 tests).
2. **Genuine fixes (test-side, low priority, NOT release-blocking the DeepSeek work):**
 - `internal/verifier`: bump the fallback-count assertion 7 → 8 (§11.4.120 reconcile).
 - `tests/performance`: restore/author `scripts/testing/competitor_speed_baseline.sh` or SKIP-OK the two competitor-baseline tests.
 - `internal/cognee`: harden `TestClientAddMemory` to not nil-deref after a failed `require.NotNil`.
3. **Investigate (UNCONFIRMED, may be REAL-BUG):** `tests/regression ConfigValidation` — print `result.Errors`; infra re-run will NOT change it.
4. **ENV/FLAKY-TIMING** (subagent, discovery, mcp, httpclient, voice, performance/scenarios, regression/TestServerStability): re-run in isolation / serialized; the divergence between the two logs confirms load sensitivity. No product action required; consider timing-tolerance hardening later.

Bottom line: NONE of the 24 failures indicates the DeepSeek default-model fix (or any product feature) is broken for the end user. The actionable code-side items are 2 stale/missing test artifacts + 1 test-robustness panic + 1 unconfirmed validator divergence. The majority (~17) are infra-absent e2e/cognee tests that need `make test-infra-up`; the remainder are host-timing flakes.